

Chapter 3: Query In Hibernate

- Hibernate Query Language
- Hibernate Criteria Queries
- Hibernate Native SQL

Hibernate Query Language

- Hibernate Query Language (HQL) is an object-oriented query language, similar to SQL, but instead of operating on tables and columns, HQL works with persistent objects and their properties.
- HQL queries are translated by Hibernate into conventional SQL queries which in turns perform action on database.
- Although you can use SQL statements directly with Hibernate using Native SQL but I would recommend to use HQL whenever possible to avoid database portability hassles, and to take advantage of Hibernate's SQL generation and caching strategies.
- Keywords like SELECT, FROM and WHERE etc. are not case sensitive but properties like table and column names are case sensitive in HQL.

FROM Clause

You will use FROM clause if you want to load a complete persistent objects into memory. Following is the simple syntax of using FROM clause:

```
String hql = "FROM Employee";  
Query query = session.createQuery(hql);  
List results = query.list();
```

If you need to fully qualify a class name in HQL, just specify the package and class name as follows:

```
String hql = "FROM com.hibernatebook.criteria.Employee";  
Query query = session.createQuery(hql);  
List results = query.list();
```

AS Clause

The AS clause can be used to assign aliases to the classes in your HQL queries, specially when you have long queries. For instance, our previous simple example would be the following:

```
String hql = "FROM Employee AS E";  
Query<Employee> query = session.createQuery(hql);  
List<Employee> results = query.list();
```

The AS keyword is optional and you can also specify the alias directly after the class name, as follows:

```
String hql = "FROM Employee E";  
Query<Employee> query = session.createQuery(hql);  
List<Employee> results = query.list();
```

SELECT Clause

The SELECT clause provides more control over the result set than the from clause. If you want to obtain few properties of objects instead of the complete object, use the SELECT clause. Following is the simple syntax of using SELECT clause to get just first_name field of the Employee object:

```
String hql = "SELECT E.firstName FROM Employee E";  
Query<Object[]> query = session.createQuery(hql);  
List< Object[]> results = query.list();
```

It is notable here that Employee.firstName is a property of Employee object rather than a field of the EMPLOYEE table.

WHERE Clause

If you want to narrow the specific objects that are returned from storage, you use the WHERE clause. Following is the simple syntax of using WHERE clause:

```
String hql = "FROM Employee E WHERE E.id = 10";  
Query<Employee> query = session.createQuery(hql);  
List<Employee> results = query.list();
```

ORDER BY Clause

To sort your HQL query's results, you will need to use the ORDER BY clause. You can order the results by any property on the objects in the result set either ascending (ASC) or descending (DESC). Following is the simple syntax of using ORDER BY clause:

```
String hql = "FROM Employee E WHERE E.id > 10 ORDER BY E.salary DESC";  
Query<Employee> query = session.createQuery(hql);  
List<Employee> results = query.list();
```

If you wanted to sort by more than one property, you would just add the additional properties to the end of the order by clause, separated by commas as follows:

```
String hql = "FROM Employee E WHERE E.id > 10 " + "ORDER BY E.firstName DESC, E.salary DESC ";  
Query<Employee> query = session.createQuery(hql);  
List<Employee> results = query.list();
```

GROUP BY Clause

This clause lets Hibernate pull information from the database and group it based on a value of an attribute and, typically, use the result to include an aggregate value. Following is the simple syntax of using GROUP BY clause:

```
String hql = "SELECT SUM(E.salary), E.lastName FROM Employee E " + "GROUP BY E.lastName";  
Query<Object[]> query = session.createQuery(hql);  
List<Object[]> results = query.list();
```

Using Named Parameters

Hibernate supports named parameters in its HQL queries. This makes writing HQL queries that accept input from the user easy and you do not have to defend against SQL injection attacks. Following is the simple syntax of using named parameters:

```
String hql = "FROM Employee E WHERE E.id = :employee_id";  
Query<Employee> query = session.createQuery(hql);  
query.setParameter("employee_id",10);  
List<Employee> results = query.list();
```

UPDATE Clause

Bulk updates are new to HQL with Hibernate 3, and deletes work differently in Hibernate 3 than they did in Hibernate 2. The Query interface now contains a method called `executeUpdate()` for executing HQL UPDATE or DELETE statements. The UPDATE clause can be used to update one or more properties of one or more objects. Following is the simple syntax of using UPDATE clause:

```
String hql = "UPDATE Employee set salary = :salary " + " WHERE id = :employee_id";
Query query = session.createQuery(hql);
query.setParameter("salary", 1000);
query.setParameter("employee_id", 10);
int result = query.executeUpdate();
System.out.println("Rows affected: " + result);
```

DELETE Clause

The DELETE clause can be used to delete one or more objects. Following is the simple syntax of using DELETE clause:

```
String hql = "DELETE FROM Employee " + "WHERE id = :employee_id";
Query query = session.createQuery(hql);
query.setParameter("employee_id", 10);
int result = query.executeUpdate();
System.out.println("Rows affected: " + result);
```

INSERT Clause

HQL supports INSERT INTO clause only where records can be inserted from one object to another object. Following is the simple syntax of using INSERT INTO clause:

```
String hql = "INSERT INTO Employee(firstName, lastName, salary)" +
"SELECT firstName, lastName, salary FROM old_employee";
Query query = session.createQuery(hql);
int result = query.executeUpdate();
System.out.println("Rows affected: " + result);
```

Aggregate Methods

HQL supports a range of aggregate methods, similar to SQL. They work the same way in HQL as in SQL and following is the list of the available functions:

1. **avg(property name)** The average of a property's value
2. **count(property name or *)** The number of times a property occurs in the results
3. **max(property name)** The maximum value of the property values
4. **min(property name)** The minimum value of the property values
5. **sum(property name)** The sum total of the property values

The distinct keyword only counts the unique values in the row set. The following query will return only unique count:

```
String hql = "SELECT count(distinct E.firstName) FROM Employee E";
Query query = session.createQuery(hql);
Long results = (Long) query.getSingleResult();
```

Pagination using Query

There are two methods of the Query interface for pagination.

1. Query **setFirstResult(int startPosition)**

This method takes an integer that represents the first row in your result set, starting with row 0.

2. Query **setMaxResults(int maxResult)**

This method tells Hibernate to retrieve a fixed number maxResults of objects.

Using above two methods together, we can construct a paging component in our web or Swing application. Following is the example which you can extend to fetch 10 rows at a time:

```
String hql = "FROM Employee";  
Query<Employee> query = session.createQuery(hql);  
query.setFirstResult(1);  
query.setMaxResults(10);  
List<Employee> results = query.list();
```

Hibernate Criteria Queries

- Hibernate provides alternate ways of manipulating objects and in turn data available in RDBMS tables. One of the methods is Criteria API which allows you to build up a criteria query object programmatically where you can apply filtration rules and logical conditions.

Following is the simplest example of a criteria query is one which will simply return every object that corresponds to the Employee class.

```
CriteriaBuilder cb = session.getCriteriaBuilder();  
CriteriaQuery<Employee> criteria = cb.createQuery(Employee.class);  
Root<Employee> root = criteria.from(Employee.class);  
criteria.select(root);  
Query<Employee> query = session.createQuery(criteria);  
List<Employee> empList = query.getResultList();
```

The above query is a simple demonstration of how to get all the items. Let's see what was done, step by step:

1. Create an instance of Session from the **SessionFactory** object
2. Create an instance of **CriteriaBuilder** by calling the **getCriteriaBuilder()** method
3. Create an instance of **CriteriaQuery** by calling the **CriteriaBuilder createQuery()** method
4. Create an instance of Query by calling the **Session createQuery()** method
5. Call the **getResultList()** method of the query object which gives us the results

Example 1: test4

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

1. test4\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test4.utils
- **HibernateUtil.java**
4. com.vedisoft.hibernate.test4.pojos
- **Employee.java** @Entity

Compiled by Rohit Ahuja

- 5. com.vedisoft.hibernate.test4.daos
 - **IEmployeeDao.java**
 - **EmployeeDao.java**
 - **IEmployeeQueryDao.java**
 - **EmployeeQueryDao.java**
- 6. com.vedisoft.hibernate.test4
 - **App.java**

Restrictions with Criteria

You can use `where()` method available for `CriteriaQuery` object to add restriction for a criteria query. Following is the example to add a restriction to return the records with salary is equal to 2000:

```
criteria.select(root).where(cb.equal(root.get("salary"), 20000));
```

Following are the few more examples covering different scenarios and can be used as per requirement:

```
// To get records having salary more than 2000
```

```
criteria.select(root).where(cb.gt(root.get("salary"), salary));
```

```
// To get records having salary less than 2000
```

```
criteria.select(root).where(cb.lt(root.get("salary"), salary));
```

```
// To get records having firstName starting with zara
```

```
criteria.select(root).where(cb.like(root.get("firstName"), "%zara%"));
```

```
// To get records having salary in between 1000 and 2000
```

```
criteria.select(root).where(cb.between(root.get("salary"), 10000, 20000));
```

```
// To check if the given property is null
```

```
criteria.select(root).where(cb.isNull(root.get("salary")));
```

```
// To check if the given property is not null
```

```
criteria.select(root).where(cb.isNotNull(root.get("salary")));
```

```
// To check if the given property is empty
```

```
criteria.select(root).where(cb.isEmpty(root.get("firstName")));
```

```
// To check if the given property is not empty
```

```
criteria.select(root).where(cb.isNotEmpty(root.get("firstName")));
```

The Criteria API allows us to easily chain expressions:

```
Predicate[] predicates = new Predicate[2];
```

```
predicates[0] = cb.isNull(root.get("itemDescription"));
```

```
predicates[1] = cb.like(root.get("itemName"), "chair%");
```

```
cr.select(root).where(predicates);
```

To add two expressions with logical operations:

```
Predicate greaterThanPrice = cb.gt(root.get("itemPrice"), 1000);
```

```
Predicate chairItems = cb.like(root.get("itemName"), "Chair%");
```

Items with the above-defined conditions joined with Logical OR:

```
cr.select(root).where(cb.or(greaterThanPrice, chairItems));
```

To get items matching with the above-defined conditions joined with Logical AND:

```
cr.select(root).where(cb.and(greaterThanPrice, chairItems));
```

Pagination using Criteria

Compiled by Rohit Ahuja

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Employee> criteria = cb.createQuery(Employee.class);
Root<Employee> root = criteria.from(Employee.class);
CriteriaQuery<Employee> select = criteria.select(root);
TypedQuery<Employee> typedQuery = sessionFactory.createEntityManager().createQuery(select);
typedQuery.setFirstResult(start);
typedQuery.setMaxResults(num);
List<Employee> empList = typedQuery.getResultList();
```

Sorting the Results

In the following example we order the list in an ascending order of the name and then in a descending order of the salary:

```
cr.orderBy(cb.asc(root.get("firstName")),cb.desc(root.get("salary")));
```

Projections & Aggregations

Get row count:

```
CriteriaQuery<Long> cr = cb.createQuery(Long.class);
Root<Employee> root = cr.from(Employee.class);
cr.select(cb.count(root));
Query<Long> query = session.createQuery(cr);
List<Long> empCount = query.getResultList();
```

The following is an example of aggregate functions:

Aggregate function for Average:

```
CriteriaQuery<Double> cr = cb.createQuery(Double.class);
Root<Employee> root = cr.from(Employee.class);
cr.select(cb.avg(root.get("EmployeePrice")));
Query<Double> query = session.createQuery(cr);
List<Double> avgEmployeeSalary = query.getResultList();
```

Other useful aggregate methods that are available are sum(), max(), min() , count() etc.

The main and most hard-hitting advantage of Criteria queries over HQL is the nice, clean, Object Oriented API.

Example 2: test5

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

1. test5\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test5.utils
- **HibernateUtil.java**
4. com.vedisoft.hibernate.test5.pojos

Compiled by Rohit Ahuja

- **Employee.java** @Entity
- 5. com.vedisoft.hibernate.test5.daos
 - **IEmployeeDao.java**
 - **EmployeeDao.java**
 - **IEmployeeQueryDao.java**
 - **EmployeeQueryDao.java**
- 6. com.vedisoft.hibernate.test5
 - **App.java**

Hibernate Native SQL

- You can use native SQL to express database queries if you want to utilize database-specific features such as query hints or the CONNECT keyword in Oracle.
- Hibernate 5.x allows you to specify handwritten SQL, including stored procedures, for all create, update, delete, and load operations.

Your application will create a native SQL query from the session with the `createNativeQuery()` method on the Session interface:

public NativeQuery createNativeQuery(String sqlString) throws HibernateException

After you pass a string containing the SQL query to the `createNativeQuery()` method, you can associate the SQL result with either an existing Hibernate entity, a join, or a scalar result using `addEntity()`, `addJoin()`, and `addScalar()` methods respectively.

Scalar queries

The most basic SQL query is to get a list of scalars (values) from one or more tables. Following is the syntax for using native SQL for scalar values:

```
String sql = "SELECT * FROM EMPLOYEE";  
NativeQuery query = session.createNativeQuery(sql);  
List<Object[]> empList = query.getResultList();  
return empList;
```

Entity queries

The above queries were all about returning scalar values, basically returning the "raw" values from the resultset.

The following is the syntax to get entity objects as a whole from a native sql query via `addEntity()`.

```
String sql = "SELECT * FROM EMPLOYEE";  
NativeQuery query = session.createNativeQuery(sql);  
query.addEntity(Employee.class);  
List<Employee> empList = query.getResultList();  
return empList;
```

Named SQL queries

The following is the syntax to get entity objects from a native sql query via `addEntity()` and using named SQL query.

```
String sql = "SELECT * FROM EMPLOYEE where firstName like :name";  
NativeQuery query = session.createNativeQuery(sql);  
query.addEntity(Employee.class);  
query.setParameter("name", "%" + name + "%");  
List<Employee> empList = query.getResultList();
```

Example 3: test6

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

1. test6 \ **pom.xml**
2. src \ main \ java \ **hibernate.cfg.xml**
3. com.vedisoft.hibernate.test6.utils
 - **HibernateUtil.java**
4. com.vedisoft.hibernate.test6.pojos
 - **Employee.java** @Entity
5. com.vedisoft.hibernate.test6.daos
 - **IEmployeeDao.java**
 - **EmployeeDao.java**
 - **IEmployeeQueryDao.java**
 - **EmployeeQueryDao.java**
6. com.vedisoft.hibernate.test6
 - **App.java**